

**ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA KỸ THUẬT MÁY TÍNH**

ĐỒ ÁN 1

**THIẾT KẾ VÀ HIỆN THỰC CƠ CHẾ XÁO TRỘN ĐỊA
CHỈ TRUY CẬP BỘ NHỚ NHẪM TĂNG CƯỜNG KHẢ
NĂNG CHỐNG TẤN CÔNG KÈ CHO NTT TRÊN FPGA**

**Design and Implementation of a Memory Address Shuffling Mechanism for Enhancing
Side-Channel Attack Resistance in FPGA-based NTT**

**GIẢNG VIÊN HƯỚNG DẪN : THS. TRƯƠNG VĂN CƯỜNG
SINH VIÊN THỰC HIỆN : NGUYỄN THANH HIẾU
MÃ SỐ SINH VIÊN : 23520486**

TP.HỒ CHÍ MINH , NĂM 2026

MỤC LỤC

CHƯƠNG 1 : GIỚI THIỆU TỔNG QUAN	2
1.1 Giới thiệu đề tài	2
1.2 Nghiên cứu liên quan.....	2
1.3 Mục tiêu.....	3
2.1. Number Theoretic Transform (NTT).....	4
2.2. Side-Channel Attack	4
2.3. Memory Address Shuffling.....	5
2.4. FPGA và Verilog HDL.....	5
2.5 Thuật toán Fish-Yates	6
3.3 NTT_core.....	10
3.4 Trace Power	11
CHƯƠNG 4 : KẾT QUẢ	12
4.1 Kết quả mô phỏng từng khối chức năng.....	12
4.1.1 Khối RPG :	12
4.1.2 Khối Address_Controller	12
4.1.3 Khối NTT_core.....	14
4.2 Kết quả mô phỏng khối Top_level.....	15
4.2.1 Pre-Synthesis	15
4.2.2 Post-Synthesis functional simulation.....	16
4.2.3 Post-Synthesis timing simulation	17
4.3 Tài nguyên sử dụng.....	21
Chương 5 : Kết luận.....	22
5.1 Kết luận.....	22
5.1.1 Ưu điểm	22
5.1.2 Nhược điểm	22
5.2 Hướng phát triển.....	22
Tài liệu tham khảo	24

CHƯƠNG 1 : GIỚI THIỆU TỔNG QUAN

1.1 Giới thiệu đề tài

Trong những năm gần đây, cùng với sự phát triển của máy tính lượng tử, nhiều thuật toán mật mã khóa công khai truyền thống như RSA và ECC được dự đoán sẽ không còn đảm bảo mức độ an toàn trong tương lai. Vì vậy, các thuật toán mật mã hậu lượng tử (PostQuantum Cryptography - PQC) đã được nghiên cứu và phát triển nhằm thay thế các hệ mật hiện nay. Trong số đó, thuật toán CRYSTALS-Kyber đã được lựa chọn trở thành tiêu chuẩn trao đổi khóa của NIST nhờ hiệu năng cao và khả năng chống lại các cuộc tấn công bằng máy tính lượng tử.

Một trong những thành phần quan trọng nhất của Kyber là Number Theoretic Transform (NTT), giúp tăng tốc các phép nhân đa thức trong miền hữu hạn. Do yêu cầu xử lý tốc độ cao, NTT thường được hiện thực trên FPGA hoặc ASIC để đáp ứng các ứng dụng nhúng và hệ thống bảo mật thời gian thực.

Tuy nhiên, mặc dù thuật toán Kyber có tính an toàn về mặt toán học, việc hiện thực trên phần cứng vẫn tồn tại nguy cơ bị khai thác thông qua Side-Channel Attack (SCA). Kẻ tấn công có thể phân tích công suất tiêu thụ, thời gian thực thi hoặc bức xạ điện từ của thiết bị để suy đoán thông tin bí mật. Đặc biệt, trong quá trình thực hiện NTT, các địa chỉ truy cập bộ nhớ thường diễn ra theo một quy luật cố định, tạo ra mẫu công suất có thể bị khai thác.

Để giảm khả năng rò rỉ thông tin này, đề tài áp dụng cơ chế Memory Address Shuffling, trong đó các địa chỉ truy cập bộ nhớ được hoán vị ngẫu nhiên trước khi đọc và ghi dữ liệu. Cơ chế này được xây dựng thông qua hai khối chính là Random Permutation Generator (RPG) và Address Controller, giúp làm thay đổi mẫu truy cập bộ nhớ nhưng vẫn đảm bảo kết quả tính toán của thuật toán NTT không bị ảnh hưởng.

1.2 Nghiên cứu liên quan

Một hướng tiếp cận phổ biến là masking, trong đó dữ liệu bí mật được che giấu bằng các giá trị ngẫu nhiên trước khi thực hiện phép tính. Phương pháp này có hiệu quả cao nhưng thường làm tăng đáng kể chi phí phần cứng, độ trễ và mức tiêu thụ năng lượng. Một hướng khác là hiding, tập trung làm giảm sự tương quan giữa dữ liệu bí mật và tín hiệu vật lý bị rò rỉ. Các kỹ thuật hiding bao gồm thêm nhiễu, cân bằng công suất, hoặc làm ngẫu nhiên trình tự thực hiện thuật toán.

Đối với NTT, nhiều nghiên cứu gần đây đã đề xuất Memory Address Shuffling, trong đó các địa chỉ truy cập bộ nhớ được hoán vị ngẫu nhiên trước khi đọc và ghi dữ liệu. Phương pháp này không làm thay đổi thuật toán NTT, chi phí phần cứng thấp và phù hợp với FPGA. Bài báo *FPGA-Friendly Random Permutation Generator for Side-Channel Resistant NTT Architectures* đã giới thiệu kiến trúc Random Permutation Generator

(RPG) kết hợp với Address Controller nhằm tạo ra các địa chỉ truy cập ngẫu nhiên trong quá trình thực hiện NTT.

Đề tài này kế thừa ý tưởng từ công trình trên và tập trung hiện thực các khối RPG, Address Controller và NTT Core bằng Verilog, đồng thời xây dựng mô hình Power Trace để đánh giá sơ bộ khả năng giảm rò rỉ thông tin.

1.3 Mục tiêu

Đề tài hướng đến việc nghiên cứu và hiện thực cơ chế shuffling địa chỉ truy cập bộ nhớ nhằm nâng cao khả năng chống tấn công kênh kề cho kiến trúc NTT trên FPGA.

Các mục tiêu cụ thể gồm:

- Nghiên cứu nguyên lý hoạt động của thuật toán Number Theoretic Transform trong Kyber.
- Thiết kế và hiện thực khối Random Permutation Generator theo kiến trúc của bài báo tham khảo.
- Thiết kế Address Controller thực hiện ánh xạ giữa địa chỉ logic và địa chỉ đã được hoán vị.
- Thiết kế NTT Core tương thích với cơ chế Address Shuffling.
- Kiểm chứng chức năng bằng Behavioral Simulation, Post-Synthesis Functional Simulation và Post-Synthesis Timing Simulation.
- Xây dựng mô hình Power Trace dựa trên Hamming Distance để đánh giá sơ bộ hiệu quả chống Side-Channel Attack.

CHƯƠNG 2 : CƠ SỞ LÝ THUYẾT

2.1. Number Theoretic Transform (NTT)

Number Theoretic Transform (NTT) là một phép biến đổi số học hoạt động trên trường hữu hạn, được xem là phiên bản rời rạc của Fast Fourier Transform (FFT). Thay vì thực hiện các phép tính trên số thực hoặc số phức, NTT thực hiện toàn bộ phép toán trong miền modulo với một số nguyên tố (q). Nhờ đó, thuật toán tránh được sai số số học và đặc biệt phù hợp với các thuật toán mật mã hiện đại.

Trong thuật toán CRYSTALS-Kyber, NTT được sử dụng để tăng tốc phép nhân đa thức. Thay vì thực hiện phép nhân trực tiếp có độ phức tạp ($O(n^2)$), các đa thức được biến đổi sang miền NTT, thực hiện phép nhân từng phần tử, sau đó biến đổi ngược trở lại miền ban đầu. Nhờ vậy, độ phức tạp được giảm xuống còn khoảng ($O(n \log n)$), giúp tăng đáng kể hiệu năng của hệ thống.

Kiến trúc NTT thường được xây dựng theo nhiều tầng (stage), trong đó mỗi tầng bao gồm nhiều phép toán Butterfly. Butterfly là đơn vị tính toán cơ bản, nhận hai phần tử đầu vào, thực hiện phép cộng và phép trừ (hoặc phép nhân với hệ số Twiddle Factor trong NTT đầy đủ), sau đó ghi kết quả trở lại bộ nhớ.

Trong quá trình thực hiện, NTT liên tục truy cập bộ nhớ theo một quy luật xác định. Chính mẫu truy cập bộ nhớ này có thể tạo ra các dấu hiệu rò rỉ về công suất tiêu thụ, từ đó trở thành mục tiêu của các cuộc tấn công kênh kề.

2.2. Side-Channel Attack

Side-Channel Attack (SCA) là phương pháp tấn công khai thác các thông tin vật lý phát sinh trong quá trình thực hiện thuật toán mật mã thay vì tấn công trực tiếp vào bản thân thuật toán. Các nguồn rò rỉ phổ biến bao gồm:

- Công suất tiêu thụ (Power Analysis).
- Thời gian thực thi (Timing Attack).

Trong số đó, phân tích công suất là phương pháp được nghiên cứu và ứng dụng nhiều nhất đối với các thiết bị phần cứng.

Một mô hình thường được sử dụng để mô phỏng công suất tiêu thụ là Hamming Distance (HD).

Hamming Distance được định nghĩa là số lượng bit thay đổi giữa hai trạng thái liên tiếp của một thanh ghi hoặc bus dữ liệu.

$$HD(x,y)=HW(x \text{ XOR } y)$$

Trong đó:

- (x): giá trị trước đó.
- (y): giá trị hiện tại.
- (HW): Hamming Weight.

Giá trị HD càng lớn thì số lượng bit chuyển đổi càng nhiều, dẫn đến công suất tiêu thụ càng lớn. Chính đặc điểm này được sử dụng để xây dựng tín hiệu power_trace trong quá trình mô phỏng.

2.3. Memory Address Shuffling

Trong kiến trúc NTT truyền thống, các địa chỉ đọc và ghi bộ nhớ được tạo theo một quy luật cố định. Điều này khiến mẫu truy cập bộ nhớ có thể bị quan sát và khai thác thông qua các cuộc tấn công Side-Channel Attack.

Để giảm khả năng rò rỉ thông tin, bài báo đề xuất kỹ thuật Memory Address Shuffling. Ý tưởng của phương pháp này là giữ nguyên dữ liệu và thuật toán NTT nhưng làm ngẫu nhiên thứ tự truy cập bộ nhớ.

Thay vì sử dụng trực tiếp địa chỉ logic do NTT Core sinh ra, các địa chỉ này sẽ được đưa qua Address Controller để ánh xạ sang các địa chỉ vật lý đã được hoán vị ngẫu nhiên.

Việc hoán vị được thực hiện bởi khối Random Permutation Generator (RPG). RPG tạo ra các chỉ số hoán vị (Permutation Index), sau đó Address Controller sử dụng các chỉ số này để chuyển đổi địa chỉ logic thành địa chỉ vật lý trước khi truy cập RAM.

Nhờ đó, mẫu truy cập bộ nhớ trở nên khó dự đoán hơn, làm giảm sự tương quan giữa công suất tiêu thụ và dữ liệu đang được xử lý, góp phần nâng cao khả năng chống lại các cuộc tấn công kênh kề.

2.4. FPGA và Verilog HDL

FPGA (Field Programmable Gate Array) là thiết bị logic khả trình cho phép người dùng hiện thực các mạch số bằng phần mềm mà không cần chế tạo phần cứng mới.

Một FPGA bao gồm nhiều tài nguyên phần cứng như:

- LUT (Look-Up Table) dùng để hiện thực các hàm logic tổ hợp.
- Flip-Flop (FF) dùng lưu trữ dữ liệu tuần tự.
- BRAM dùng lưu trữ dữ liệu dung lượng lớn.
- DSP Slice hỗ trợ các phép nhân và cộng tốc độ cao.

Trong đề tài này, toàn bộ hệ thống được thiết kế bằng ngôn ngữ mô tả phần cứng Verilog HDL và được tổng hợp trên phần mềm Vivado.

Quy trình thiết kế bao gồm:

- Thiết kế RTL bằng Verilog.
- Behavioral Simulation để kiểm tra chức năng.
- Synthesis nhằm chuyển RTL thành cổng logic.
- Post-Synthesis Functional Simulation để kiểm tra sau tổng hợp.
- Post-Synthesis Timing Simulation để đánh giá ảnh hưởng của độ trễ phần cứng.

2.5 Thuật toán Fish-Yates

Là một thuật toán để xáo trộn một chuỗi hữu hạn. Thuật toán này lấy một danh sách tất cả các phần tử của chuỗi và liên tục xác định phần tử tiếp theo trong chuỗi đã được xáo trộn bằng cách rút ngẫu nhiên một phần tử từ danh sách cho đến khi không còn phần tử nào. Thuật toán này tạo ra một hoán vị không thiên vị : mọi hoán vị đều có khả năng xảy ra như nhau.

```

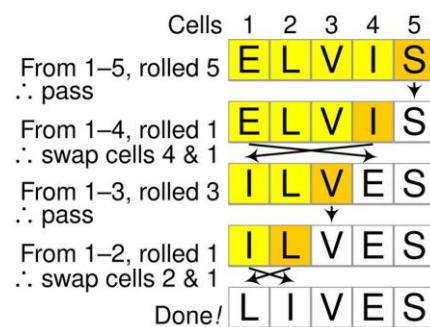
REG ← {0b, 0c, ..., 32, 33, 3f, 3e, ..., 3a, 39,
       0a, 09, ..., 01, 00, 38, 37, 36, 35, 34}
FIFO :ena ← 1          for i = 0 to 63 do          for j = 0 to 5 do
BUF[j] ← LFSR          end for                    FIFOin ← BUF
end for                for k = 0 to 11 do
    rest ← 63 - k
    idx 0 ← FIFOout > 0x28 ? FIFOout : FIFOout - 0x28
    FIFOin ← REG[idx 0]    REG[idx 0] ← REG[rest]
    end for                for k = 12 to 63 do

        rest ← 63 - k
        idx 1 ← FIFOout > rest ? FIFOout : FIFOout & rest
        FIFOin ← REG[idx 1]

        REG[idx 1] ← REG[rest]
    end for                for i = 0 to 5 do

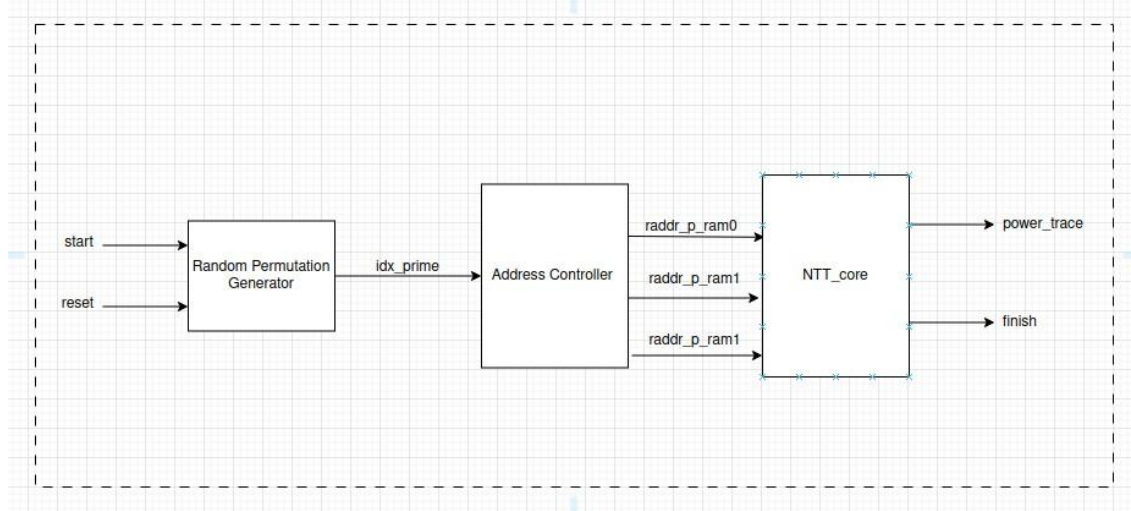
        FIFOin ← FIFOout
    end for
    FIFOena ← 0

```



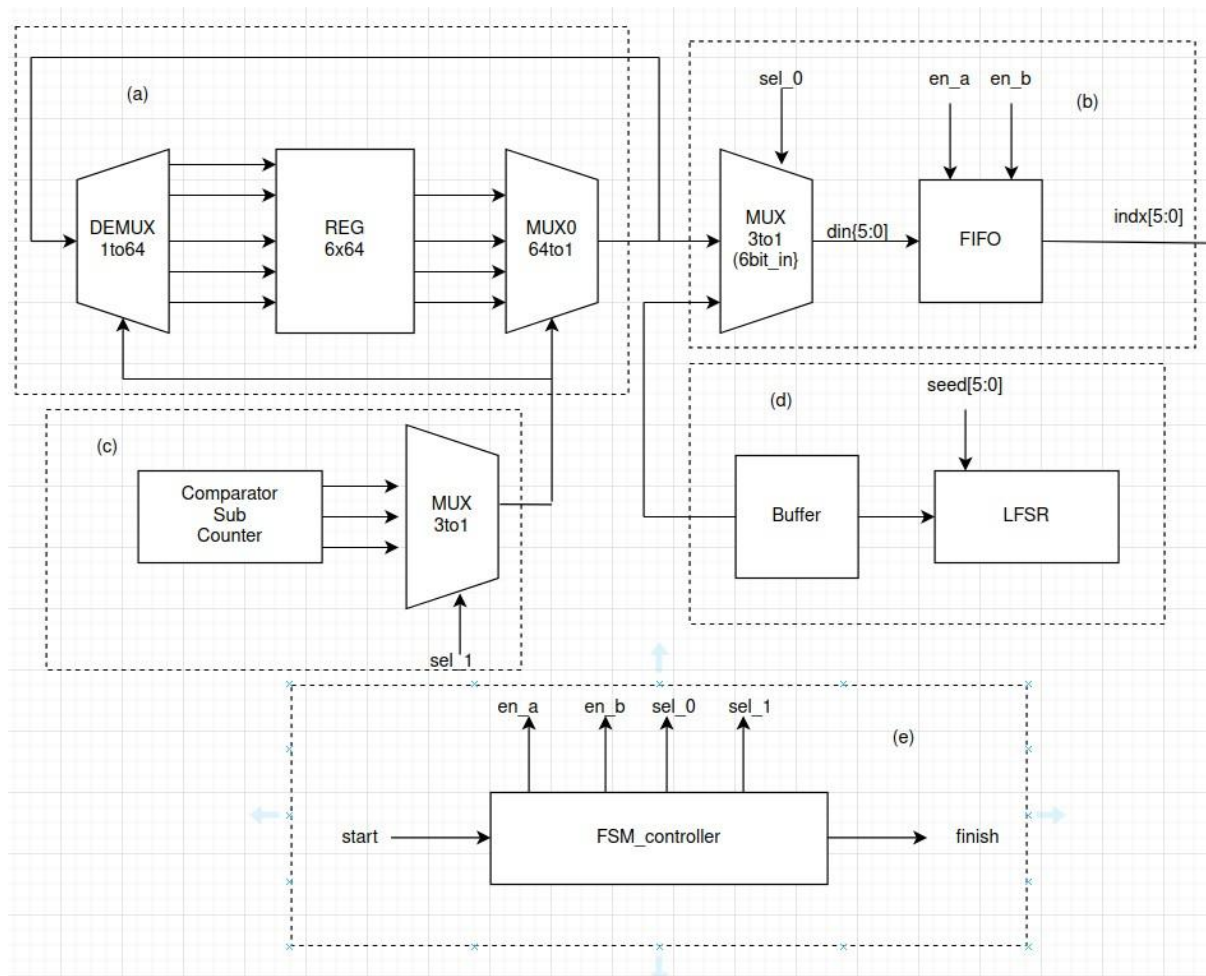
CHƯƠNG 3 : MÔ HÌNH THIẾT KẾ

-Tổng quan thiết kế:



- Mô hình thiết kế tổng thể của hệ thống là một kiến trúc xử lý toán học an toàn phần cứng (Secure NTT Processor Architecture), được tối ưu hóa cho thuật toán mã hóa hậu kỳ lượng tử Kyber nhằm mục đích chống lại các cuộc tấn công kênh kề (SCA). Hệ thống vận hành theo một luồng đường ống tuyến tính khép kín từ trái sang phải, bắt đầu bằng việc nhận các tín hiệu điều khiển toàn cục là start và reset tại khối Random Permutation Generator (RPG).
- Tại đây, khối RPG đóng vai trò là nguồn mật mã, chịu trách nhiệm sinh ra chuỗi hoán vị giả ngẫu nhiên và xuất ra dưới dạng chỉ số `idx_prime`. Chỉ số này ngay sau đó được đẩy vào khối trung gian Address Controller — một bộ lọc bảo mật thông minh có nhiệm vụ tiếp nhận `idx_prime` cùng các tín hiệu trạng thái và địa chỉ gốc, thực hiện kéo trễ đường ống, nâng cấp không gian bit và băm trộn để tạo ra các bus địa chỉ an toàn bao gồm `raddr_p_ram0`, `raddr_p_ram1` và địa chỉ ghi.
- Cuối cùng, các đường địa chỉ đã được xáo trộn này đóng vai trò làm đầu vào trực tiếp cho khối đích NTT_core. Lõi xử lý trung tâm này sẽ thực thi máy trạng thái tuần hoàn để truy xuất dữ liệu RAM, thực hiện phép toán bướm toán học, đồng thời xuất ra tín hiệu cờ báo hoàn thành `finish` và vết năng lượng `power_trace` (tính theo khoảng cách Hamming) giúp đánh giá trực quan khả năng kháng tấn công của toàn hệ thống. Với tư duy thiết kế phân tách độc lập giữa luồng bảo mật và luồng tính toán, mô hình này tạo nên một IP Core hoàn chỉnh, có tính tùy biến cao và khả năng bảo vệ chủ động cực kỳ hiệu quả.

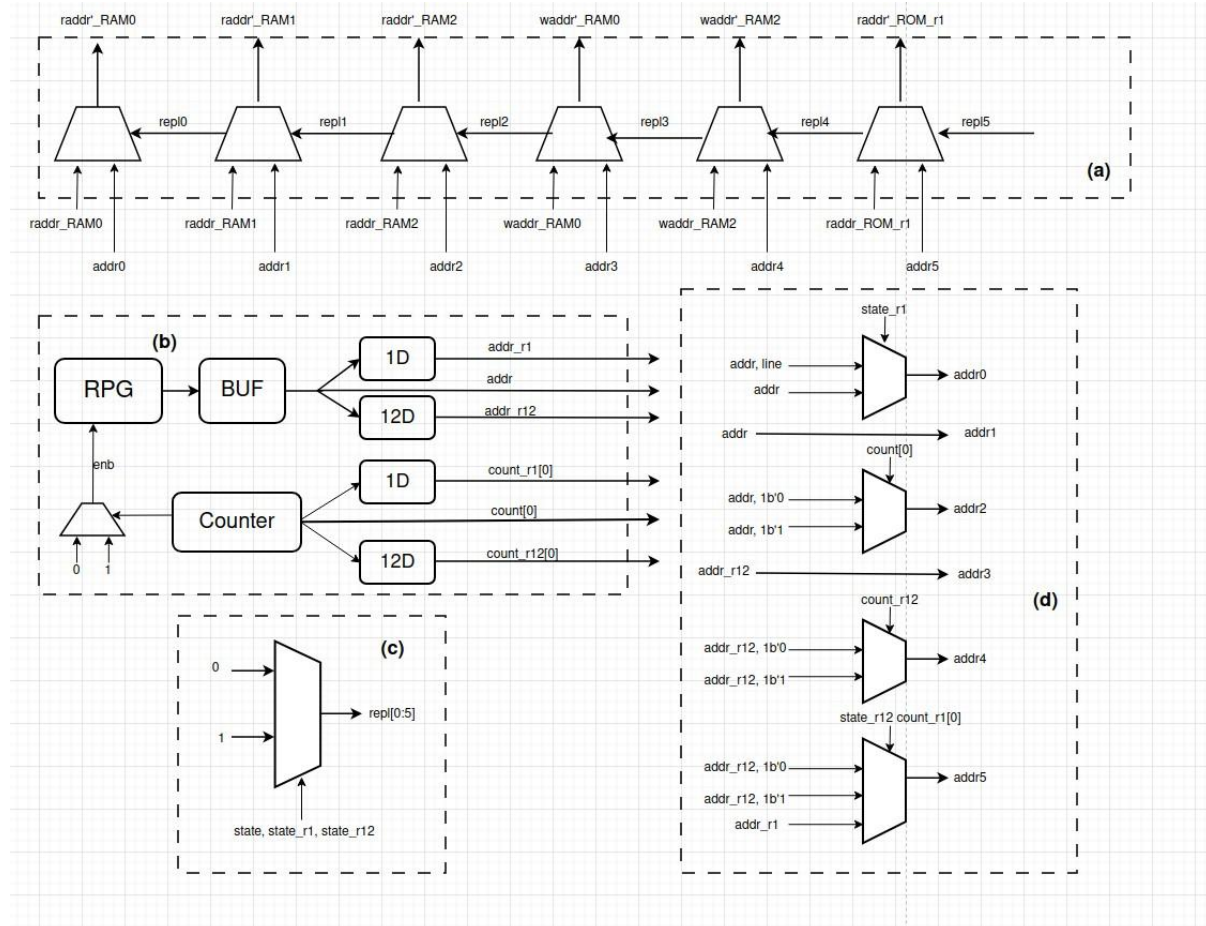
3.1 Random permutation generate -RPG:



- Bắt đầu từ khối LFSR, nơi liên tục sinh ra các bit ngẫu nhiên dựa trên giá trị seed ban đầu. Các bit ngẫu nhiên này được đưa vào khối buffer để gom nhiều bit 1-bit thành một chỉ số ngẫu nhiên 6-bit (rand_idx). Sau mỗi sáu chu kỳ clock, buffer xuất ra một giá trị ngẫu nhiên hoàn chỉnh và phát tín hiệu valid để báo dữ liệu hợp lệ.
- Chỉ số ngẫu nhiên sau đó được đưa qua MUX1 trước khi ghi vào FIFO. Tùy theo tín hiệu điều khiển sel0, dữ liệu đưa vào FIFO có thể là giá trị ngẫu nhiên mới từ LFSR hoặc là giá trị permutation đã được lưu trước đó trong bộ nhớ REG. FIFO hoạt động ở hai chế độ: chế độ input mode để nhận dữ liệu mới từ LFSR hoặc REG, và chế độ cyclic mode để tuần hoàn lại dữ liệu bên trong FIFO nhằm giảm tài nguyên phần cứng. Hai tín hiệu ena và enb được controller sử dụng để điều khiển hai chế độ hoạt động này.
- Dữ liệu đầu ra của FIFO được chuyển đến khối index_control, nơi các chỉ số ngẫu nhiên được điều chỉnh để phù hợp với hai giai đoạn shuffling của kiến trúc. Khối này tạo ra giá trị idx, chính là địa chỉ hoán vị cuối cùng được dùng để truy cập bộ

nhớ permutation trong phần REG của RPG. REG lưu toàn bộ bảng hoán vị và sử dụng MUX0 cùng DEMUX để đọc và cập nhật các phần tử permutation tương ứng với idx

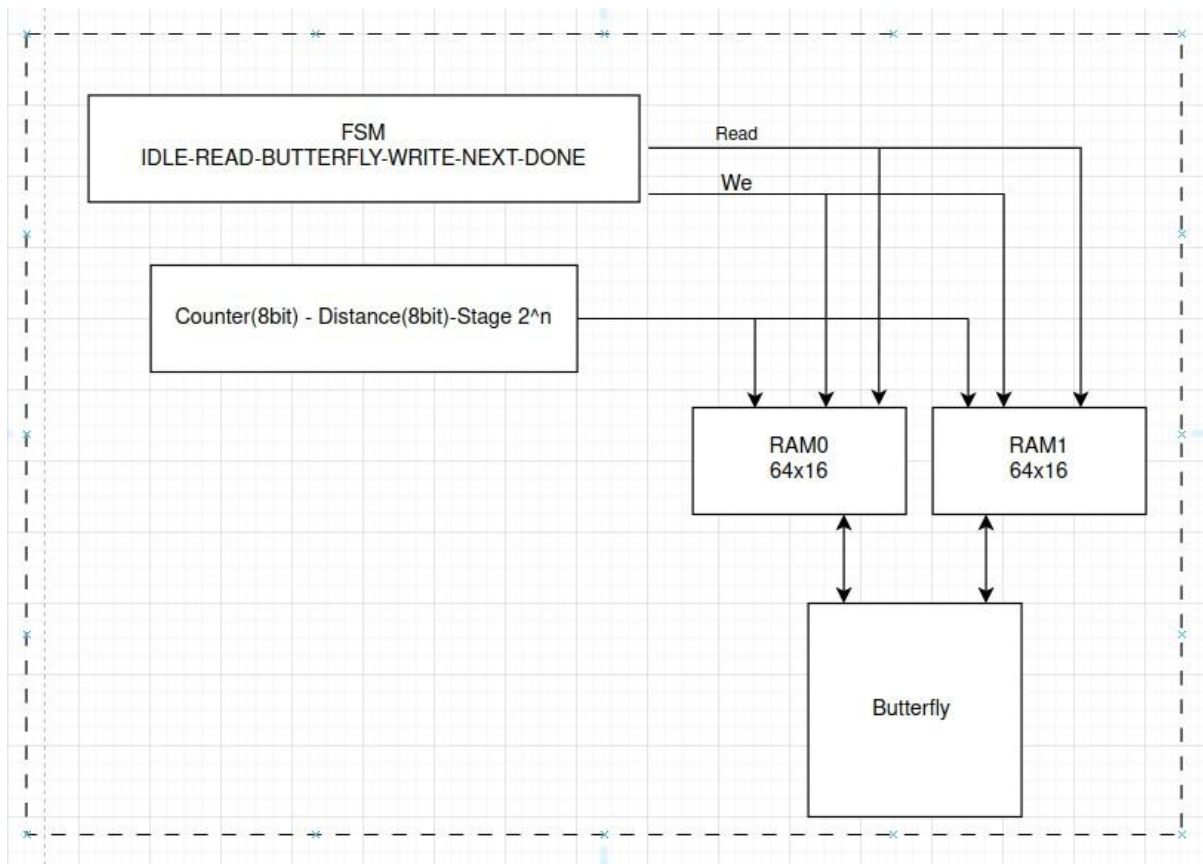
3.2 Address Controller



- Khối Address Controller có nhiệm vụ chuyển đổi các địa chỉ tuần tự do NTT_core tạo ra thành các địa chỉ đã được xáo trộn bởi RPG. Đường đi dữ liệu bắt đầu từ đầu vào `addr_in`, là giá trị permutation (`idx_prime`) do RPG cung cấp. Giá trị này được đưa vào phần B của Address Controller để tạo ra các phiên bản delay khác nhau như `addr_r1` và `addr_r12` thông qua các shift register 1 chu kỳ và 12 chu kỳ. Đồng thời, một bộ đếm nội bộ cũng tạo ra các tín hiệu `count`, `count_r1`, và `count_r12` nhằm đồng bộ hóa các stage xử lý của NTT.
- Trong phần D, các giá trị permutation được ghép thêm bit hoặc kết hợp với tín hiệu `line` để tạo thành sáu địa chỉ xáo trộn mới `addr0` đến `addr5`
- Song song với đó, phần C của Address Controller sinh các tín hiệu điều khiển `repl0` đến `repl5` dựa trên trạng thái hoạt động của NTT như `pwm` và `sub`. Các tín hiệu này quyết định thời điểm cần bật hoặc tắt cơ chế xáo trộn địa chỉ.

- Cuối cùng, trong phần A, sáu bộ MUX 2-to-1 được sử dụng để lựa chọn giữa địa chỉ gốc từ NTT_core và địa chỉ đã được xáo trộn từ addr0~addr5

3.3 NTT_core



-Ý tưởng : Khối **NTT_core** được thiết kế nhằm mô phỏng datapath của thuật toán Number Theoretic Transform (NTT) trong Kyber, đồng thời tích hợp cơ chế shuffling địa chỉ để phục vụ Side-Channel Attack – SCA.

-Mục tiêu chính của thiết kế không phải tối ưu hiệu năng tính toán NTT hoàn chỉnh mà tập trung vào:

- tạo mẫu truy cập bộ nhớ giống NTT thực,
- kết hợp cơ chế random permutation,
- mô phỏng leakage trên đường truy cập RAM

-Cách hoạt động :

FSM trong NTT_core tạo ra: raddr_RAM0 và raddr_RAM1 -đây là các địa chỉ truy cập tuần tự của butterfly trong NTT.

Các địa chỉ gốc không được dùng trực tiếp để truy cập RAM.

Thay vào đó:

- Address Controller nhận `idx_prime`
 - Ánh xạ lại thành: `raddr_p_RAM0` và `raddr_p_RAM1` là các địa chỉ đã được xáo trộn.
- Sau khi có địa chỉ shuffled:

`A <= RAM0[raddr_p_RAM0];`

`B <= RAM1[raddr_p_RAM1];`

hai toán hạng butterfly được lấy từ RAM.

Thiết kế hiện tại dùng butterfly đơn giản nhằm:

- Giảm độ phức tạp,
- Tập trung vào memory access leakage

3.4 Trace Power

Trong thiết kế này, khối `power_trace` được sử dụng để mô phỏng rò rỉ công suất (power leakage) của phần cứng

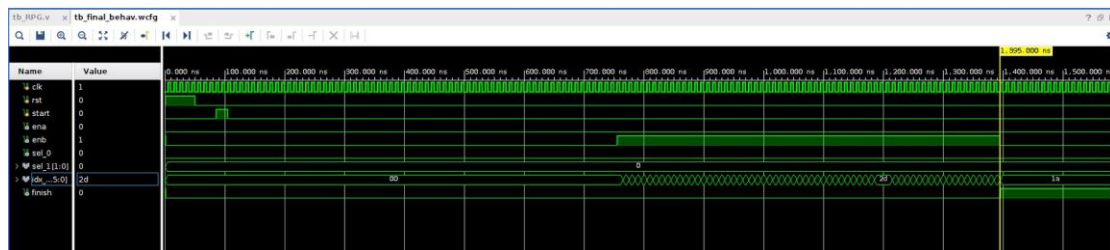
Mô hình này giả sử: Power n số bit thay đổi giữa hai trạng thái liên tiếp. Khi trạng thái đang là read thì lúc này sẽ tìm ra sự thay đổi vì: leakage mạnh nhất thường xảy ra lúc truy cập RAM và memory decoder switching.

Do đó: chỉ trace ở READ stage để mô phỏng realistic hơn.

CHƯƠNG 4 : KẾT QUẢ

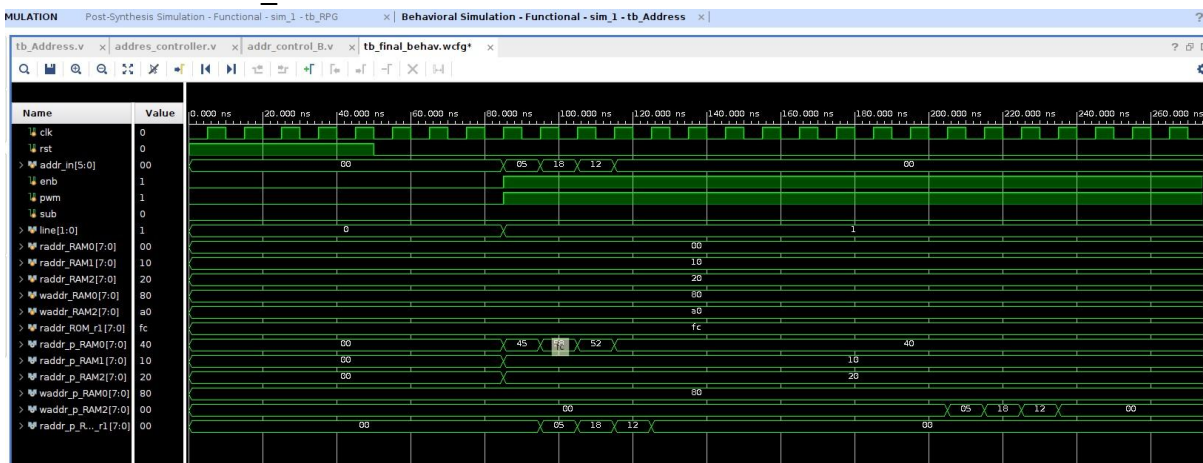
4.1 Kết quả mô phỏng từng khối chức năng

4.1.1 Khối RPG :



- Đầu tiên thiết lập chuỗi Reset kéo dài (5ns +50ns reset + 30ns đợi + 20ns cấp start) giúp đẩy thời điểm mạch bắt đầu xử lý thực sự (start = 1) ra xa mốc 0ns đầu tiên, là khoảng thời gian khởi tạo phần cứng
- Chu trình này thường mất nhiều chu kỳ clock để quét qua các ô nhớ ban đầu. Mỗi chu kỳ là 10 ns, mạch chỉ cần chạy khoảng 64 chu kỳ nạp (từ 115ns đến 755 ns) và chuẩn bị là đồng hồ đã nhảy lên mốc .
- Từ 755 ns có tín hiệu enb =1 thì sinh ra 64 giá trị idx_prime

4.1.2 Khối Address_Controller



Trạng thái của các tín hiệu điều khiển đầu vào

- enb = 1 và pwm = 1 được bật lên đồng thời tại mốc 80 ns.
- line[1:0]: Ban đầu ở mức 0, đến mốc 80 ns chuyển sang 1 (giá trị nhị phân 01 tương ứng với thiết lập trong testbench).

Các địa chỉ gốc đầu vào ($raddr_RAM0 = 00$, $raddr_RAM1 = 10$, $raddr_RAM2 = 20...$) được giữ cố định chuỗi giá trị 05 => 18 => 12 được cấp tại $addr_in$ từ mốc 80ns:

$raddr_p_ROM_r1$ (Ngõ ra cuối cùng):

Nhìn vào đường cuối cùng, tại mốc 90 ns, giá trị 05 xuất hiện. Tại 100 ns là 18, và tại 110 ns là 12.

Đường này trễ chính xác 1 chu kỳ clock so với $addr_in$. Điều này chứng tỏ ngõ ra này lấy trực tiếp giá trị từ thanh ghi đệm đầu tiên ($addr / addr_buf$) của khối B.

$waddr_p_RAM2$:

Khoảng thời gian từ 80 ns (lúc vào) đến 210 ns (lúc ra) chính là $210 - 80 = 130$ ns trễ đúng 13 chu kỳ clock). Đây chính là kết quả đi xuyên qua mạng dịch 12 tầng $addr_delay12$ từ khối B truyền tới khối A

Khối $addr_control_A$ (U9) có nhiệm vụ: Nếu điều kiện thay thế thỏa mãn ($repl = 1$), nó sẽ lấy địa chỉ đã hoán vị ép ra ngõ ra. Nếu không, nó giữ nguyên địa chỉ gốc. Hãy nhìn vào các ngõ ra RAM để thấy rõ sự hoán đổi này:

$raddr_p_RAM0$:

Từ 80 ns đến 110 ns, ngõ ra này liên tục biến đổi thành các giá trị 45 => 58 => 52. Đây chính là các địa chỉ xáo trộn được tính toán từ khối D ($addr0 - addr5$) dựa trên đầu vào $addr_in$.

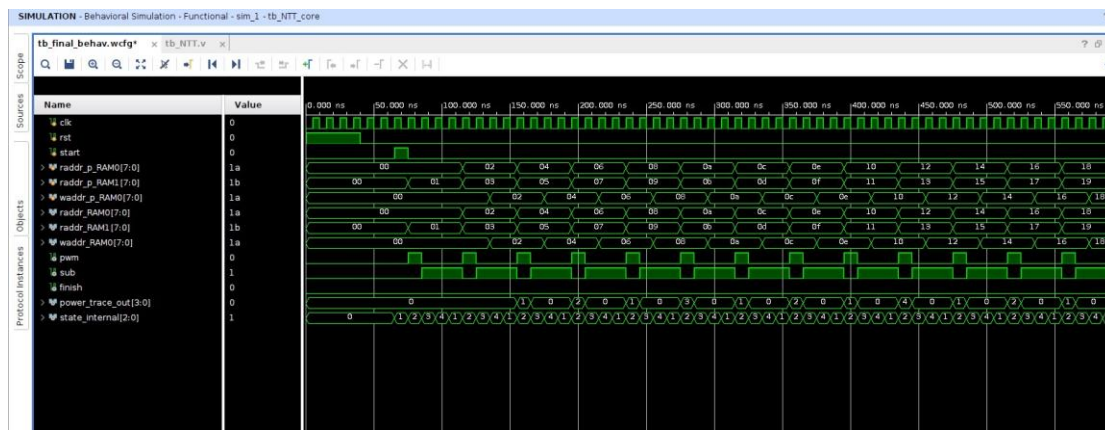
Sau mốc 170 ns, khi chuỗi dữ liệu ngẫu nhiên đã đi qua hết, ngõ ra này quay trở về giá trị mặc định ổn định là 40 (hệ Hex).

$raddr_p_RAM1$ và $raddr_p_RAM2$:

Giữ nguyên giá trị 10 và 20 xuyên suốt mạch. Điều này chứng tỏ tại thời điểm này, máy trạng thái quyết định giữ nguyên địa chỉ RAM1 và RAM2 gốc mà không hoán đổi chúng.

$waddr_p_RAM0$: Giữ nguyên giá trị 80 (bằng đúng địa chỉ ghi gốc $waddr_RAM0$).

4.1.3 Khối NTT_core



Chu trình chuyển trạng thái (FSM) tuần hoàn 4 bước

Nhìn xuống đường `state_internal[2:0]` ở dưới cùng, sẽ thấy sau khi xung `start` kích hoạt, mạch lặp đi lặp lại chuỗi số:

- 1 (READ): Chu kỳ đọc dữ liệu từ RAM.
- 2 (BUTTERFLY): Thực hiện phép toán bướm ($A+B$ và $A-B$).
- 3 (WRITE): Ghi kết quả ngược lại vào RAM.
- 4 (NEXT): Tăng bộ đếm địa chỉ ($\text{counter} \leq \text{counter} + 2$) và kiểm tra điều kiện lặp để quay về trạng thái READ (1).

Mỗi chu trình lớn mất đúng 4 chu kỳ xung clock (40 ns) để hoàn thành một cặp tính toán.

Vì đang ở giai đoạn đầu của thuật toán NTT, khoảng cách distance đang bằng 1. Do đó:

- Ở trạng thái READ, raddr_RAM0 nhận giá trị của counter (tăng tiến theo các số chẵn: 00, 02, 04, 06, 08, 0a...).
- Chân raddr_RAM1 luôn đi trước raddr_RAM0 đúng 1 đơn vị vị trí (counter + distance), tạo thành chuỗi số lẻ tương ứng: 01, 03, 05, 07, 09, 0b...
- Đến chu kỳ WRITE (3), chân waddr_RAM0 cập nhật lại đúng địa chỉ chẵn để ghi kết quả add result vào RAM0.

Tín hiệu điều khiển pwm và sub

Hai tín hiệu này hoạt động như một cặp xung lệch pha nhịp nhàng điều khiển phản ứng:

- $pwm = 1$ chỉ bật lên duy nhất tại trạng thái READ (1) để báo hiệu kích hoạt bộ nhân/xử lý giai đoạn đầu (hoặc đồng bộ hóa với hệ thống ngoài). Nhìn trên xung, cứ đến state 1 là pwm dừng đứng lên 1 chu kỳ clock.

- sub = 1 bật lên ngay sau đó tại trạng thái BUTTERFLY (2) nhằm kích hoạt tính toán trừ/phép toán bướm giai đoạn sau và kéo dài qua trạng thái WRITE trước khi bị xóa ở trạng thái NEXT (4).

Đường power_trace_out chỉ xuất hiện giá trị (khác 0) tại trạng thái READ (1), đúng theo logic tính khoảng cách Hamming của bus địa chỉ đọc raddr_p_RAM0.

Sự thay đổi các bit giữa 2 chu kỳ đọc liên tiếp để hiểu tại sao nó ra các con số 1, 2, 3, 4:

Mốc 130 ns (Giá trị xuất hiện = 1):

Địa chỉ trước đó: 00 (nhị phân: 0000_0000)

Địa chỉ hiện tại: 02 (nhị phân: 0000_0010)

Số bit thay đổi (XOR): Chỉ có bit số 1 thay đổi (\$0 \rightarrow 1\$) $\rightarrow$ power_trace_out = 1.

Mốc 170 ns (Giá trị xuất hiện = 2):

Địa chỉ trước đó: 02 (nhị phân: 0000_0010)

Địa chỉ hiện tại: 04 (nhị phân: 0000_0100)

Số bit thay đổi: (2 bit thay đổi) power_trace_out = 2.

Mốc 250 ns (Giá trị xuất hiện = 3):

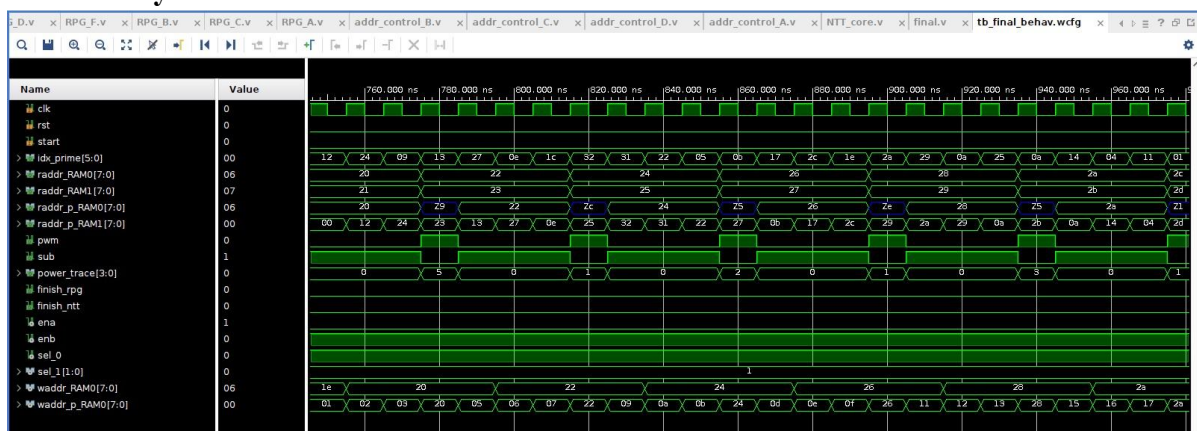
Địa chỉ trước đó: 06 (nhị phân: 0000_0110)

Địa chỉ hiện tại: 08 (nhị phân: 0000_1000)

Số bit thay đổi: power_trace_out = 3.

4.2 Kết quả mô phỏng khối Top_level

4.2.1 Pre-Synthesis



-Giải thích idx_prime[5:0] thay đổi liên tục: 12 $\rightarrow$ 24 $\rightarrow$ 09 $\rightarrow$ 13 $\rightarrow$ 27 $\rightarrow$...

Đây là:

- Chỉ số permutation được sinh bởi RPG,
- Dùng để random hóa địa chỉ truy cập RAM.

Các tín hiệu: raddr_p_RAM0 và raddr_p_RAM1 đã được ánh xạ lại bởi Address Controller.

pwm = 1 xuất hiện trong: READ stage. sub = 1 xuất hiện trong: BUTTERFLY stage.

power_trace[3:0] : XOR địa chỉ shuffled hiện tại và địa chỉ shuffled trước đó.

Giả sử:

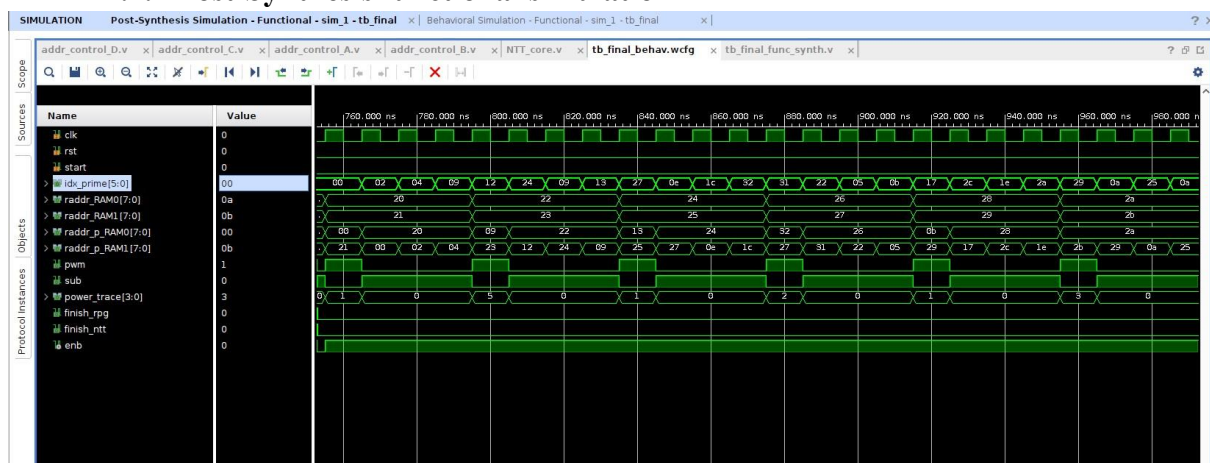
current=20 = 0010_0000

previous= 29 = 0010_1001

XOR: 0000_1001

Có: 2 bit đổi trạng thái → switching activity tăng.

4.2.2 Post-Synthesis functional simulation



Tín hiệu raddr_RAM0 và raddr_RAM1 là các địa chỉ truy cập gốc được tạo bởi khối NTT Core. Có thể thấy các địa chỉ này tăng tuần tự theo cặp butterfly:

(20,21) → (22,23) → (24,25) → (26,27) → ...

Đây là mẫu truy cập đặc trưng của thuật toán NTT truyền thống.

Sau khi đi qua khối Address Controller, các địa chỉ được ánh xạ lại thành raddr_p_RAM0 và raddr_p_RAM1 dựa trên chỉ số hoán vị idx_prime do RPG sinh ra. Quan sát waveform cho

thấy các địa chỉ sau shuffling không còn tuân theo quy luật tăng tuần tự mà thay đổi theo chuỗi giả ngẫu nhiên:

20 → 09 → 22 → 13 → 24 → 32 → ...

và

21 → 00 → 02 → 04 → 23 → 12 → ...

Điều này chứng tỏ cơ chế shuffling đã làm thay đổi đường truy cập bộ nhớ trong khi vẫn giữ nguyên chức năng tính toán của NTT.

Tín hiệu `idx_prime` liên tục thay đổi theo các giá trị:

00 → 02 → 04 → 09 → 12 → 24 → ...

Cho thấy bộ sinh hoán vị RPG đang hoạt động bình thường và liên tục cung cấp các chỉ số hoán vị cho Address Controller.

Các tín hiệu điều khiển `pwm` và `sub` xuất hiện tuần hoàn theo từng chu kỳ butterfly. Trong đó `pwm` được kích hoạt ở pha đọc dữ liệu từ RAM, còn `sub` được kích hoạt ở pha thực hiện phép toán butterfly.

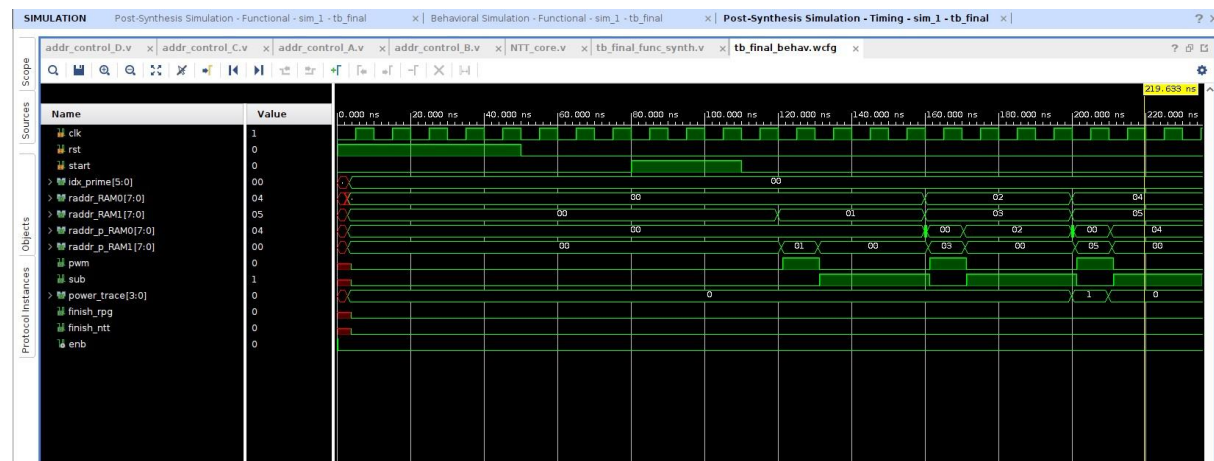
Để đánh giá khả năng chống rò rỉ thông tin, một mô hình công suất đơn giản dựa trên Hamming Distance được xây dựng thông qua tín hiệu `power_trace`. Giá trị của tín hiệu này thay đổi theo:

1 → 5 → 1 → 2 → 1 → 3 → ...

Tín hiệu `power_trace` biểu diễn số bit thay đổi giữa hai địa chỉ truy cập liên tiếp

4.2.3 Post-Synthesis timing simulation

Từ 0 ns => 220ns



Từ 0 ns → 50 ns rst = 1 và start = 0

Lúc này toàn bộ hệ thống bị reset.

Do đó: Các giá trị X xuất hiện là hoàn toàn bình thường trong Timing Simulation.

Nguyên nhân:

- Các FF vừa được khởi tạo.
- Delay sau synthesis làm tín hiệu chưa ổn định ngay lập tức.

Khi ~50 ns:

Sau khi: rst = 0

Các thanh ghi bắt đầu nhận giá trị xác định.

Ta thấy: idx_prime = 00, raddr_RAM0 = 00, raddr_RAM1 = 00

Hệ thống đang ở trạng thái IDLE.

-Start hệ thống (~80 ns)

Tín hiệu: start = 1

Xuất hiện trong một khoảng ngắn.

FSM của NTT Core chuyển: IDLE=>READ=>BUTTERFLY=>WRITE=>NEXT

-Khoảng: 120 ns → 160 ns

Ta thấy: raddr_RAM0 = 00, raddr_RAM1 = 01

NTT Core đọc cặp dữ liệu đầu tiên.

Đồng thời: raddr_p_RAM0 = 00, raddr_p_RAM1 = 01

Vì lúc này RPG mới khởi động nên địa chỉ shuffled vẫn gần giống địa chỉ gốc.

-Sau khoảng: 160 ns

- Sau synthesis, tín hiệu phải đi qua LUT.
- LUT sinh ra delay vài trăm ps đến vài ns.

Do đó:

raddr_RAM0 thay đổi => tính toán => raddr_p_RAM0 cập nhật

nên xuất hiện hiện tượng lệch pha nhỏ.

Đây là hành vi bình thường của Timing Simulation.

FSM vẫn chạy theo chuỗi: READ => BUTTERFLY => WRITE => NEXT

không bị thay đổi sau synthesis.

Power Trace

Quan sát: 0 → 2 → 1 → 3 → 1 → 2 → 4 → 1

Power Trace vẫn thay đổi liên tục.

Điều này chứng tỏ:

- Địa chỉ truy cập RAM thay đổi theo thời gian.
- Mẫu công suất giả lập trở nên khó dự đoán.

-Kết quả cho timing

Design Timing Summary			
Setup		Hold	Pulse Width
Worst Negative Slack (WNS):		Worst Hold Slack (WHS):	Worst Pulse Width Slack (WPWS):
3.248 ns		0.134 ns	4.500 ns
Total Negative Slack (TNS):		Total Hold Slack (THS):	Total Pulse Width Negative Slack (TPWS):
0.000 ns		0.000 ns	0.000 ns
Number of Failing Endpoints:		Number of Failing Endpoints:	Number of Failing Endpoints:
0		0	0
Total Number of Endpoints:		Total Number of Endpoints:	Total Number of Endpoints:
1721		1721	878
All user specified timing constraints are met.			

$$\text{công thức } F_{\max} = \frac{1}{T_{\min}} = \frac{1}{T_{\text{current}} - \text{WNS}} = 148,1 \text{ MHz}$$

T_{current} : chu kì clk hiện tại = 10 ns

WNS = 3,248 ns

4.3 Tài nguyên sử dụng

Name	1	Slice LUTs (17600)	Slice Registers (35200)	F7 Muxes (8800)	F8 Muxes (4400)	Bonded IOB (54)	BUFGCTRL (32)
▼ N final		453	877	91	43	49	1
> ADDR0 (addres_controller)		0	6	0	0	0	0
NTT0 (NTT_core)		49	48	0	0	0	0
> RPG0 (top)		404	823	91	43	0	0

Số lượng tài nguyên phần cứng thu được thấp hơn đáng kể so với bài báo gốc do NTT Core trong nghiên cứu này được đơn giản hóa nhằm tập trung đánh giá cơ chế shuffling địa chỉ và ảnh hưởng đến khả năng chống Side-Channel Attack. Các khối tính toán phức tạp như Montgomery Multiplier, Barrett Reduction và kiến trúc NTT đầy đủ của Kyber chưa được hiện thực. Do đó, kết quả tổng hợp chủ yếu phản ánh chi phí phần cứng của cơ chế RPG và Address Controller thay vì toàn bộ gia tốc Kyber NTT.

Chương 5 : Kết luận

5.1 Kết luận

5.1.1 Ưu điểm

Tính hoàn chỉnh và đồng bộ cao của kiến trúc (Top-level Integration): Đề tài đã hiện thực hóa thành công một hệ thống bảo mật phần cứng hoàn chỉnh theo đúng mô hình đường ống tuyến tính từ RPG → Address Controller → NTT_core.

Việc xây dựng thành công bộ đo khoảng cách Hamming (power_trace_out) . Nó giúp đo mức độ rò rỉ năng lượng trực quan ngay trên phần mềm mô phỏng (Vivado Simulation)

5.1.2 Nhược điểm

Gia tăng độ trễ tổng thể (Latency): Việc đi qua các khối trung gian để băm trộn địa chỉ và việc duy trì đường ống sâu vô hình trung làm tăng thời gian trễ trước khi cập kết quả tính toán đầu tiên được ghi lại vào bộ nhớ. Hệ thống sẽ có thời gian thực thi (Execution Time) dài hơn một chút so với các lõi NTT thuần túy không có tính năng bảo mật.

Mô hình rò rỉ mới ở mức giả lập: Khối power_trace hiện tại mới chỉ tính toán dựa trên mô hình Khoảng cách Hamming của Bus địa chỉ đọc. Trên chip thực tế, rò rỉ năng lượng còn đến từ Bus dữ liệu, mạch logic của bộ cộng/trừ và nhiều nền, do đó mô hình này cần được mở rộng để tiệm cận hơn với thực tế

5.2 Hướng phát triển

Chia sẻ tài nguyên (Resource Sharing): Tối ưu hóa các khối Part B, C, D của bộ Address Controller để có thể tái sử dụng chung các bộ đếm hoặc mạch logic tính toán, giúp giảm diện tích thiết kế (Area Overhead) khi tổng hợp (Synthesis) lên chip.

Mở rộng bộ giám sát rò rỉ: Nâng cấp khối power_trace_out để không chỉ tính Khoảng cách Hamming trên bus địa chỉ đọc, mà còn tính toán trên cả Bus dữ liệu (Data Bus) và Trạng thái bên trong của bộ tính toán toán học (ALU - bộ cộng/trừ bướm).

Tích hợp cơ chế Masking: Kết hợp phương pháp xáo trộn địa chỉ (Shuffling) hiện tại với kỹ thuật che giấu dữ liệu toán học (Arithmetic/Boolean Masking) để tạo ra màn hình bảo vệ kép, giúp lõi NTT kháng được cả các cuộc tấn công kênh kề bậc cao (Higher-Order SCA).

Đa dạng hóa thuật toán RPG: Thiết kế thêm các chế độ sinh số ngẫu nhiên khác nhau cho khối RPG (ví dụ tích hợp PRNG dựa trên LFSR cải tiến hoặc TRNG - Bộ sinh số ngẫu nhiên thực tế từ nhiễu phần cứng) để tăng tính hỗn loạn và khó đoán của chuỗi chỉ số idx_prime.

Triển khai trên FPGA/ASIC: Tiến hành nạp bo mạch thực tế (chạy Post-Implementation) trên các dòng chip FPGA (như Xilinx Artix-7/Zynq) để đo đạc chính xác tần số hoạt động cực đại (F_{max}), công suất tiêu thụ thực tế và diện tích lát cắt tài nguyên (LUTs/FFs).

Tài liệu tham khảo

- [1] National Institute of Standards and Technology (NIST), *Module-Lattice-Based KeyEncapsulation Mechanism Standard (ML-KEM)*, FIPS 203, Aug. 2024.
- [2] Xilinx Inc., *Vivado Design Suite User Guide: Synthesis (UG901)*, 2024.
- [3] "A Hardware-Friendly Shuffling Countermeasure Against Side-Channel Attacks for Kyber," IEEE Conference/Journal Paper